

Part 3

```
import socket

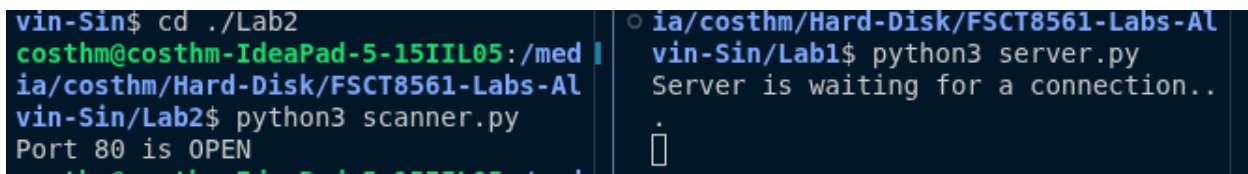
# TARGET = "127.0.0.1"
# PORT = 80

# scanner_socket = socket.socket(
#     socket.AF_INET,
#     socket.SOCK_STREAM
# )

# scanner_socket.settimeout(0.5)

# result = scanner_socket.connect_ex(
#     (TARGET, PORT)
# )

# if result == 0:
#     print("Port", PORT, "is OPEN")
# else:
#     print("Port", PORT, "is not open")
```



The image shows a terminal window with two panes. The left pane shows the execution of a scanner script: `vin-Sin$ cd ./Lab2`, `costhm@costhm-IdeaPad-5-15IIL05:/media/costhm/Hard-Disk/FSC8561-Labs-All$ python3 scanner.py`, and the output `Port 80 is OPEN`. The right pane shows the execution of a server script: `ia/costhm/Hard-Disk/FSC8561-Labs-All$ python3 server.py`, with the output `Server is waiting for a connection..` and a cursor.

Part 4

```
def scan_port(target, port):
    sock = socket.socket(
        socket.AF_INET,
        socket.SOCK_STREAM
    )

    socket.setdefaulttimeout(0.5)

    result = sock.connect_ex(
        (target, port)
    )
```

```
sock.close()

if result == 0:
    return True
else:
    return False

print(scan_port("127.0.0.1", 80))
```

```
ia/costhm/Hard-Disk/FSCI8561-Labs-AL
vin-Sin/Lab2$ python3 scanner.py
True
```

Part 5

```
def scan_port(target, port):
    sock = socket.socket(
        socket.AF_INET,
        socket.SOCK_STREAM
    )

    socket.setdefaulttimeout(0.5)

    result = sock.connect_ex(
        (target, port)
    )

    sock.close()

    if result == 0:
        return True
    else:
        return False

target = "127.0.0.1"
open_port = []

for port in range(1, 1025):
    if scan_port(target, port):
        open_port.append(port)

print("Open ports:", open_port)
```

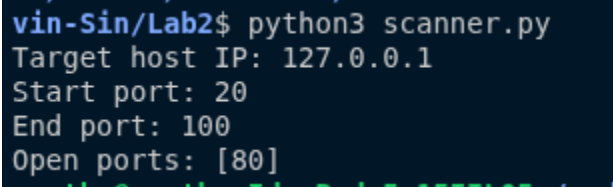
```
vin-Sin/Lab2$ python3 scanner.py
Open ports: [80, 631, 902]
```

Part 7

```
# Modified code (with same function)
target = input("Target host IP: ")
start_port = input("Start port: ")
end_port = input("End port: ")
open_port = []

for port in range(int(start_port), int(end_port)):
    if scan_port(target, port):
        open_port.append(port)

print("Open ports:", open_port)
```



```
vin-Sin/Lab2$ python3 scanner.py
Target host IP: 127.0.0.1
Start port: 20
End port: 100
Open ports: [80]
```

Part 8

```
# same function modified code
target = input("Target host IP: ")
start_port = int(input("Start port: "))
end_port = int(input("End port: "))

max_port = 1000

if start_port < 1 or start_port > 65535:
    print("Invalid start port. Port must be between 1 and 65535.")
elif end_port < 1 or end_port > 65535:
    print("Invalid end port. Port must be between 1 and 65535.")
elif start_port > end_port:
    print("Invalid port range. Start port must be less than or equal to end port.")
elif end_port - start_port + 1 > max_port:
    print(f"Port range is too large. You can scan a maximum of {max_port} ports at once.")
else:
    open_port = []
    for port in range(start_port, end_port + 1):
        if scan_port(target, port):
            open_port.append(port)
```

```
print("Open ports:", open_port)
```

```
costhm@costhm-IdeaPad-5-15IIL05:/media/costhm/Hard-Disk/FSCT8561-Labs-AL
● vin-Sin/Lab2$ python3 scanner.py
Target host IP: 127.0.0.1
Start port: 1
End port: 2000
Port range is too large. You can scan a maximum of 1000 ports at once.
```

Part 9

```
def scan_port(target, port):
    sock = socket.socket(
        socket.AF_INET,
        socket.SOCK_STREAM
    )

    try:
        sock.settimeout(0.5)

        result = sock.connect_ex(
            (target, port)
        )

        return result == 0 # if 0 returns true

    finally:
        sock.close

try:
    target = input("Target Host: ") # prompt for host

    try:
        target_ip = socket.gethostbyname(target) # resolve for hostname ip
        print(f"Resolved {target} to {target_ip}")

    except socket.gaierror:
        print("Bad hostname or IP addr")
        exit() # go to the resolve for hostname

    start_port = int(input("Start Port: ")) # prompt for ports
    end_port = int(input("End Port: "))
```

```

max_port = 1000

if start_port < 1 or start_port > 65535:
    print("Invalid start port. Port must be between 1 and 65535.")
elif end_port < 1 or end_port > 65535:
    print("Invalid end port. Port must be between 1 and 65535.")
elif start_port > end_port:
    print("Invalid port range. Start port must be less than or equal to end port.")
elif end_port - start_port + 1 > max_port:
    print(f"Port range is too large. You can scan a maximum of {max_port} ports at once.")
else: # Real deal
    open_port = [] # adding to open port list if found any
    for i in range(start_port, end_port + 1):
        if scan_port(target_ip, i):
            open_port.append(i)

    if open_port: # when list aint empty
        print("\nOpen ports: ")

        for i in open_port:
            try:
                service = socket.getservbyport(i, "tcp") # i as port, specify tcp
            except OSError: # handler if service is returned as error
                service = "unknown"
            print(f"Port {i}: {service}")
        else:
            print("\nNo open ports detected")

except ValueError: # invalid input handle crash
    print("bad input, please be smart and type in numbers")
except KeyboardInterrupt:
    print("\nNot doing scan anymore")

```

```

in/Lab2$ python3 scanner.py
Target Host: 127.0.0.1
Resolved 127.0.0.1 to 127.0.0.1
Start Port: 1
End Port: 1000

Open ports:
Port 80: http
Port 631: ipp
Port 902: unknown

```

Part 10

```
costhm@costhm-IdeaPad-5-15IIL05: /media/costhm/Hard-Disk/FSC78561-Labs-Alvin-S
• in/Lab2$ python3 scanner.py
Target Host: g
Bad hostname or IP addr
costhm@costhm-IdeaPad-5-15IIL05: /media/costhm/Hard-Disk/FSC78561-Labs-Alvin-S
• in/Lab2$ python3 scanner.py
Target Host: 127.0.0.1
Resolved 127.0.0.1 to 127.0.0.1
Start Port: 0
End Port: 2
Invalid start port. Port must be between 1 and 65535.
costhm@costhm-IdeaPad-5-15IIL05: /media/costhm/Hard-Disk/FSC78561-Labs-Alvin-S
• in/Lab2$ python3 scanner.py
Target Host: 127.0.0.1
Resolved 127.0.0.1 to 127.0.0.1
Start Port: 1
End Port: 65536
Invalid end port. Port must be between 1 and 65535.
costhm@costhm-IdeaPad-5-15IIL05: /media/costhm/Hard-Disk/FSC78561-Labs-Alvin-S
• in/Lab2$ python3 scanner.py
Target Host: 127.0.0.1
Resolved 127.0.0.1 to 127.0.0.1
Start Port: 55
End Port: 33
Invalid port range. Start port must be less than or equal to end port.
```

Demonstrated error handling for bad hostname, port lower or bigger than the possible range (1 or 65535) and start port being larger than the end port

Part 11

```
• $ python3 scanner.py
Target Host: 127.0.0.1
Resolved 127.0.0.1 to 127.0.0.1
Start Port: 1
End Port: 1000

Open ports:
Port 80: http
Port 631: ipp
Port 902: unknown
```

```
costhm@costhm-IdeaPad-5-15IIL05:~$ nmap -p 1-1000 127.0.0.1
Starting Nmap 7.94SVN ( https://nmap.org ) at 2026-09-29 04:43 PDT
Nmap scan report for localhost (127.0.0.1)
Host is up (0.00020s latency).
Not shown: 997 closed tcp ports (conn-refused)
PORT      STATE SERVICE
80/tcp    open  http
631/tcp   open  ipp
902/tcp   open  iss-realsecure

Nmap done: 1 IP address (1 host up) scanned in 0.09 seconds
```

1. Yes it did, within the given range
2. Time it took, when the scan took place, showing hostname for ip provided, and how many number of ports are filtered out
3. Probably not
4. To learn what is inside of nmap, learning about sockets and proper error handling

Part 12

Part 1 to 5

```
Labs-Alvin-Sin/Lab2$ python3 namp_scanner.py
['127.0.0.1']
['tcp']
{80: {'state': 'open', 'reason': 'syn-ack', 'name': 'http', 'product': 'Caddy
httpd', 'version': '', 'extrainfo': '', 'conf': '10', 'cpe': 'cpe:/a:matt_ho
lt:caddy'}}
```

Part 6

```
def main():
    # 1. Ask for target
    target = input("Target host: ").strip()

    if not target:
        print("Error: Target cannot be empty.")
        return

    # 2. Ask for start and end port
    try:
        start_port = int(input("Start port: "))
        end_port = int(input("End port: "))

    # 3. Validate input
    except ValueError:
        print("Error: Ports must be valid numbers.")
        return
```

```

if start_port < 1 or start_port > 65535:
    print("Error: Start port must be between 1 and 65535.")
    return

if end_port < 1 or end_port > 65535:
    print("Error: End port must be between 1 and 65535.")
    return

if start_port > end_port:
    print("Error: Start port must be less than or equal to end port.")
    return

# Limit the scan range
max_ports = 1000

if end_port - start_port + 1 > max_ports:
    print(f"Error: You can scan a maximum of {max_ports} ports at once.")
    return

# Create Nmap scanner
scanner = nmap.PortScanner()

port_range = f"{start_port}-{end_port}"

print(f"\nScanning {target} ports {port_range}...")
print("-" * 50)

# 4. Perform scan using python-nmap
try:
    scanner.scan(
        target,
        port_range,
        arguments="-sT -sV"
    )

except nmap.PortScannerError as error:
    print(f"Scan error: {error}")
    return

except Exception as error:
    print(f"Unexpected error: {error}")
    return

# Check whether the target was found
if target not in scanner.all_hosts():
    print("No scan results were returned.")
    return

# 5. Display discovered ports

```

```

found_ports = []

for host in scanner.all_hosts():

    print(f"Target: {host}")
    print(f"State: {scanner[host].state()}")
    print()

    if "tcp" in scanner[host]:
        for port in sorted(scanner[host]["tcp"]):

            # 6. Display state
            state = scanner[host]["tcp"][port]["state"]

            # 7. Display service information
            service = scanner[host]["tcp"][port].get("name", "")
            product = scanner[host]["tcp"][port].get("product", "")
            version = scanner[host]["tcp"][port].get("version", "")

            found_ports.append(port)

            print(f"Port: {port}")
            print(f"State: {state}")
            print(f"Service: {service or 'Unknown'}")

            if product:
                print(f"Product: {product}")

            if version:
                print(f"Version: {version}")

            print("-" * 50)

# 9. Clear result when no ports are discovered
if not found_ports:
    print("No ports were discovered in the specified range.")

if __name__ == "__main__":
    main()

```

```

$ python3 nmap_scanner.py
Port: 80
State: open
Service: http
Product: Caddy httpd
-----
Port: 631
State: open
Service: ipp
Product: CUPS
Version: 2.4
-----
Port: 902
State: open
Service: vmware-auth
Product: VMware Authentication Daemon
Version: 1.10
-----

```

1. Which version required more code?

The raw socket version requires more code as we had to create the socket, set timeout, connect to each port, check the result then close the socket. On top of handling scanning logic ourselves. The python-nmap version was shorter because nmap library handles most of the scanning work.

2. Which version provided more information?

The python-nmap version provided more information, displaying the port state, service name, and product when it is available

3. What information did Nmap provide that your raw socket scanner did not?

Nmap provided service detection and version information, it could identify a port as running HTTPs and provide information about it. The raw socket scanner mainly determines if a TCP port is open or not

4. What work is **python-nmap hiding from the programmer?**

Python-nmap hides much of the scanning and result processing work. Nmap handles creating and sending network probes, communication with ports, determining port states, detecting services and version and processing the scan results.

5. Why was it useful to build the raw socket version before using **python-nmap?**

To understand what is happening during a TCP port scan and learn how sockets are created, how a connection attempt can mean if a port is open, how timeout work, why sockets need to be closed and error handling. This further allows us to understand how nmap work fundamentally

6. If you were developing a real security application, which approach would you choose and why?

For a real work scenario, using nmap is probably a better option as it provides way more functions than just a simple scanner, plus it is more optimized than python. Does not matter what the situation is, nmap is just a better choice overall if i have to ever do probing, port scanning, or any work related to nmap, just use nmap.

Part 13

1. **What information can an attacker learn from port scanning?**
They can learn which ports are open, what services may be running, and potentially which applications or versions are exposed.
2. **Why can an open port increase a system's attack surface?**
An open port provides a way for network traffic to reach a service, creating another potential entry point that could be attacked.
3. **Does an open port automatically mean that a vulnerability exists? Explain.**
No. An open port only means that a service is accepting connections. The service may be properly secured and have no known vulnerabilities.
4. **Why should unnecessary network services be disabled or restricted?**
It reduces the number of exposed services and therefore reduces the system's potential attack surface.
5. **What is the difference between an open port and a closed port?**
An **open port** has a service accepting connections. A **closed port** is reachable, but no service is accepting connections on it.
6. **Why might a firewall make scan results difficult to interpret?**
A firewall can block, reject, or silently drop connection attempts, making it difficult to determine whether a port is actually closed or simply being filtered.
7. **Why can our simple TCP Connect scanner not reliably identify every filtered port?**
It only relies on whether a TCP connection succeeds. If a firewall silently drops the connection, the scanner may simply receive a timeout and cannot determine exactly why the connection failed.
8. **How could defenders detect or limit scanning activity?**
They can use firewalls, intrusion detection/prevention systems, network monitoring, rate limiting, and logging to detect or restrict unusual scanning activity.
9. **Why should defenders scan their own systems?**
Scanning helps identify unnecessarily exposed ports and services so defenders can secure them before they are discovered and exploited by unauthorized users.
10. **Why is authorization important before performing network scanning?**
Scanning systems without permission can violate policies or laws and may disrupt services. Authorization ensures the scan is performed within an approved scope.

Part 14

The scan was performed against my own local machine. I scanned multiple TCP ports to identify services that were accessible from the scanning system. The scan discovered the following open ports: **80, 631, 902 in range of 1 - 1000**. The associated services identified by Nmap included **http, ipp, iss-realsecure**. These results show which network services are currently exposed and potentially reachable by other systems on the network.

From a defender's perspective, an open port does not automatically indicate that the system is vulnerable. However, every exposed service increases the system's attack surface because it provides another network-accessible point that could potentially be targeted. For example, if a port is not required, leaving it accessible could provide unnecessary exposure. Services that are required should still be kept updated, properly configured, and restricted to only the networks or users that need access.

There were also noticeable differences between the Python raw socket scanner and Nmap. The Python scanner primarily determined whether a TCP connection could be established, so its results were limited to identifying reachable/open ports. Nmap provided additional information, including service names and, when available, product and version information. This made the Nmap results more useful for understanding what was actually running behind the open ports. The raw socket scanner was useful for understanding the basic TCP connection process, while Nmap automated much of the scanning and service-detection work.

To reduce unnecessary exposure, a defender could disable services that are not needed and use host-based or network firewalls to restrict access to required services. Services should also be regularly patched and configured securely. Network monitoring and intrusion detection could help identify unexpected scanning or connection attempts. Overall, the scan provided a useful snapshot of the target's exposed network services and showed why defenders should regularly review which ports are accessible and whether each exposed service is actually necessary.